SSTables and LSM-Trees:

"sorted string table" → SSTable

- Keys in index are sorted

Since incoming writes can be in any order, keep a balanced tree data structure in memory (AVL tree or Red-Black tree), with these trees, you can insert keys in any order and read them back in sorted order

memtable

When the memtable's size surpasses some threshold, write it out to disk as a SSTable file.

- new file becomes most recent segment file
- writes continue to a new memtable instance during this process

Segment files:

- created when the memtable gets too large

Merging Segments:

1. read files side-by-side and look at the first key in each file

2. copy the lowest key (according to sorted order) to the new segment file, repeat

↳ if same key appears in multiple segments, copy from the most recent segment, and discard the other instances of the key in older segments

(each segment file has its own sparse index)

Sparse in-memory index:

- one key for every few kilobytes of data

Ex. If looking for "handiwork", but dont know exact byte offset of that key in segment file. However, you know the offset for "handbag" and "handsome".

This what the index tracks:
Key: byte offset

Since sorted, "handiwork" between those two: start at "handbag" offset, search until "handsome" offset.

Block Compression:

- Reads must scan over several KV pairs in the requested range (between "x" and "y").
- So, group those records into a block and compress before writing to disk
- Now, each entry in sparse-index points to the start of a block

LSM-Tree :

Is an algorithmic architecture that :

1. adds incoming writes to a memtable
2. when memtable gets big enough, it is written to disk
3. to serve reads, first check memtable and then each segment in order of how recent it is
4. Periodically run a merging and compaction process in the background

To prevent data loss on crash, keep a WAL on disk. This log can be used to restore the memtable

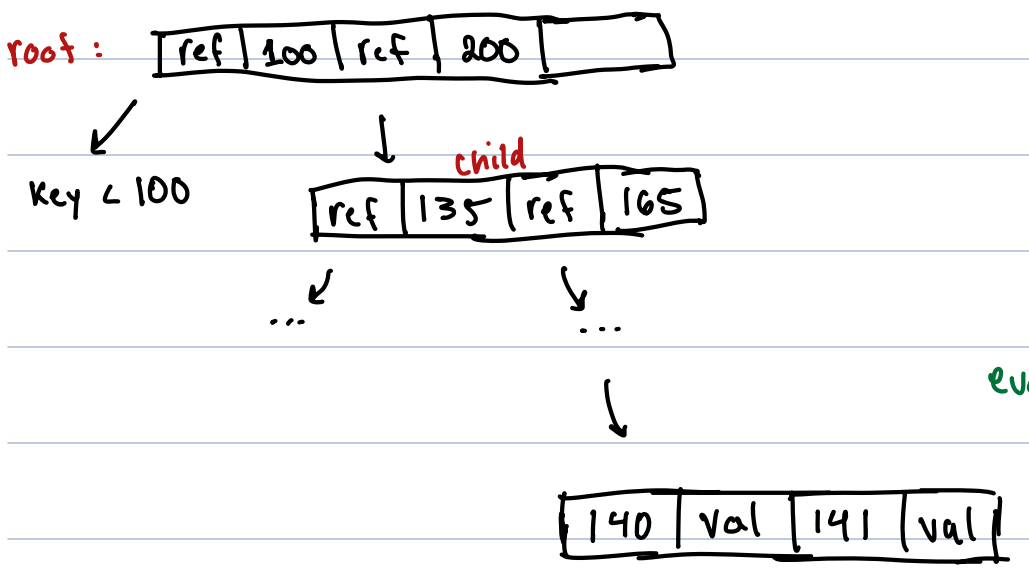
B-Trees:

- break the DB down into fixed size pages, typically 4-8 KB in size
- read/write one page at a time

Each page has its own address, allowing one page to reference another, similar to a pointer, but on disk instead of memory

One page designated as the root, all queries start here.

- contains several keys and references to child pages
- each child responsible for a continuous range of keys
- keys between the references indicate where the boundaries between those ranges lie



eventually reach "leaf" pages that contain individual keys and their value or references to where it can be found

branching factor - # of references to child pages in one page

↓ of a B-tree

typically several hundred

Writes:

update:

- search for leaf page containing the key, change the value and write page back to disk

insertion:

- find leaf page whose range encompasses the new key and add it to the page
- if page does not have enough space, it is split into two half-full pages, and the parent page is updated to reflect this split of a single page into two pages

This ensures that the tree remains BALANCED

depth = $O(\log n)$

Add a WAL to make the B-tree resilient to crashes

Some implementations try to lay out the tree so that leaf

pages appear in sequential order on disk to reduce disk seek time ⟶ this is difficult to maintain

LSM-Trees vs B-Trees:

As a rule of thumb:

- LSM-trees faster for writes
- B-Trees faster for reads

Write amplification - one write to DB results in multiple writes to the disk over the lifetime of the DB

LSM-Tree Advantages:

- higher write throughput because they sequentially write compact SSTables (much faster than random writes)
- can be compressed better

LSM-Tree Disadvantages:

- compaction process can interfere with performance of ongoing reads and writes

Other Indexing Structures:

Storing values within the index:

- value in an index can either be:
  1. actual row in question
  2. reference to row stored elsewhere
     - In this case, the place the row is stored is called the **heap file** → good when you have multiple secondary indexes; no data duplication

↙ sometimes, extra hop from index to heap file is too much a performance penalty on reads

clustered index - row is stored directly within an index (ex: primary key of a table)

covering index - stores some of the tables columns in the index

Keeping everything in memory:

- **in-memory databases**
- some (like Memcached) are intended for caching only, as data is lost if machine is restarted, others aim for durability using logs, snapshots, or replication
- faster because they avoid the over-head of encoding in-memory datastructures in a form that can be written to

disk

Transaction Processing or Analytics:

- need different systems for OLTP and OLAP because of the very different query patterns

OLAP data model:

Star Schema

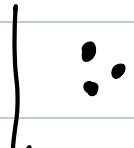
- center of the schema is the **Fact Table**; contains events that occured at a particular time
- some of the columns are attributes, some are FK's to other tables called **dimension tables**

↓

represent 5 w's

Snowflake Schema - dimensions are further broken down into sub-dimensions

For a query you usually only need the data from four or five of the hundreds of columns that exist in the table



Column Orientated Storage:

- store all the values from one column together; each column file contains the rows in the same order so we can recreate a specific record by taking the value at the same position in each column file
- only load columns from disk that are required by the query

Column Compression

- easy to compress column-orientated data
- often, # of distinct values in a column is small compared to the # of columns → billions of transactions, only 100,000 products

bitmap encoding - take a column with n distinct values and and turn it into n seperate bitmaps; one bitmap for each distmct value, one bit for each row

Ex.

Column values:

val: 10 10 9 6 5 5 9

Bitmaps:

val = 10: 1 1 0 0 0 0 0

val = 9: 0 0 1 0 0 0 1

val = 6: 0 0 0 1 0 0 0

⋮ etc

run-length encoding: Can use this when bitmaps are very large

val = 10: 0, 2 → 2 zeros, 2 ones, rest zeros

val = 9: 2, 1, 3, 1 → 2 zeros, 1 one, 3 zeros, 1 one, rest zeros

val = 6: 3, 1

⋮ etc

With bitmaps you can use bitwise OR / AND for queries which is very efficient

Sort Order in Column Storage:

- can't sort each column independently because then you don't know which items in the columns belong to the same row
- data needs to be sorted an entire row at a time
- can also sort each replica of the data on different machines in different ways, then use the version that best fits your query

Writes :

- use LSM-Trees